



Assignment 1

Computernetze und ihre Leistung

von:

Michael Heise

Linus Henn

Luca Lars Stoevers

Matrikelnr.: 537385

Matrikelnr.: 576038

Matrikelnr.: XXXXXX

Kurs:

Computernetze und ihre Leistung SoSe 2026

Dozent: **Ralph-Günther Holz**

Münster, 29. Mai 2026

Contents / Inhaltsverzeichnis

1	HTTP/1 und HTTP/2	1
1.1	Aufbau der Abfragen	1
1.2	Methodik zur Analyse der Abfragen	2
1.3	Analyse der HTTP/2-Kommunikation	3
1.4	Analyse der HTTP/1.1-Kommunikation	6
1.5	Vergleich der HTTP/1.1- und HTTP/2-Austausche	7
1.6	Ablaufbeschreibung eines HTTP/2-Clients über h2c	9
1.7	Reflexion und Diskussion	9
2	SMTP	11

1 HTTP/1 und HTTP/2

Abweichend von der Reihenfolge der Aufgabenstellung ist die Methodik nach dem Aufbau beschrieben. Dies sorgt für ein klareres Verständnis, wie wir zu unseren Ergebnissen gekommen sind.

1.1 Aufbau der Abfragen

Für alle Anfragen wurde ein einfaches Python Script verwendet, welches die TCP-Verbindung aufbaut, die HTTP-Anfrage sendet und die Antwort empfängt 1.

```
1 import asyncio
2 import httpx
3 import argparse
4
5 HTTP_1 = "1"
6 HTTP_2 = "2"
7
8 async def send_http_request(host, port, path, HTTP_version=HTTP_1):
9     http_1 = HTTP_version == HTTP_1
10    http_2 = HTTP_version == HTTP_2
11    async with httpx.AsyncClient(http1=http_1, http2=http_2) as client:
12        response = await client.get(f"http://{host}:{port}{path}")
13        print(f"{response.http_version} {response.status_code} {response.
14              reason_phrase}")
15        print("Headers:")
16        for header, value in response.headers.items():
17            print(f"  {header}: {value}")
18        print(f"Response Data: {response.text}")
19
20 if __name__ == "__main__":
21     parser = argparse.ArgumentParser()
22     parser.add_argument("--host", default="cuil.internet-transparency.org",
23                         help="Host to connect to")
24     parser.add_argument("--port", type=int, default=80, help="Port to
25                       connect to")
26     parser.add_argument("--path", default="/index.html", help="Path to
27                       request")
28     parser.add_argument("--http_version", choices=[HTTP_1, HTTP_2], default
29                         =HTTP_1, help="HTTP version to use (1 or 2)")
30     args = parser.parse_args()
31     asyncio.run(send_http_request(args.host, args.port, args.path, args.
32                                 http_version))
```

Code 1: Python Script zum Aufbau von TCP-Verbindungen und Durchführung von HTTP-Anfragen

Das Script nutzt für die Anfragen die `httpx` Bibliothek [1], welche die TCP-Verbindung für HTTP-Anfragen vereinfacht. Es werden die HTTP-Versionen 1.1 und 2 unterstützt, welche über die Kommandozeile ausgewählt werden können 2. Das Script gibt die Antwort des Servers auf der Kommandozeile aus.

```

1 usage: connection.py [-h] [--host HOST] [--port PORT] [--path PATH]
2                       [--http_version {1,2}]
3
4 options:
5   -h, --help            show this help message and exit
6   --host HOST            Host to connect to
7   --port PORT            Port to connect to
8   --path PATH            Path to request
9   --http_version {1,2}  HTTP version to use (1 or 2)

```

Code 2: Ausgabe der Python Script Optionen über den Befehl `python connection.py --help`

1.2 Methodik zur Analyse der Abfragen

Die HTTP-Kommunikation zwischen dem Client und dem Server wurde über Wireshark analysiert. Dabei wurden die folgenden Schritte durchgeführt:

1. Starten von Wireshark und Auswahl der entsprechenden Netzwerkschnittstelle, um den Datenverkehr zu erfassen.
2. Anfrage an die Webseite mit dem Python Skript 1 senden, um die HTTP-Kommunikation zu initiieren.

```
python3 connection.py --http-version ...
```

3. Aufgezeichnete Pakete nach der Anfrage durchsuchen.
4. Filterung der Frame Pakete Nummern um nur DNS-Abfrage, TCP-Handshake und HTTP-Pakete zu betrachten.

```
frame.number >= {ErstesPaketNummer} && frame.number <= {LetztesPaketNummer}
```

5. Speichern der relevanten Pakete für die weitere Analyse.
6. Analyse der HTTP-Pakete, um die Kommunikation zwischen Client und Server zu verstehen, einschließlich der Anfragen und Antworten, die über HTTP gesendet wurden.

Zur Analyse der Kommunikation wurden einige Filterbefehle in Wireshark verwendet, um die relevanten Pakete zu identifizieren und zu isolieren.

1. `frame.number >= {ErstesPaketNummer} && frame.number <= {LetztesPaketNummer}`: Dieser Filter wurde verwendet, um die Pakete zu isolieren, die während der HTTP-Kommunikation zwischen Client und Server ausgetauscht wurden. Dabei wurden die Paketnummern des ersten und letzten relevanten Pakets angegeben, um nur diese Pakete anzuzeigen und nur diese in die Datei zu exportieren.
2. `http` und `http2`: Dieser Filter wurde verwendet, um nur die HTTP-Pakete anzuzeigen, die während der Kommunikation ausgetauscht wurden. Dadurch konnten die HTTP-Anfragen und -Antworten isoliert und analysiert werden, ohne von anderen Paketen wie DNS oder TCP-Handshake Paketen abgelenkt zu werden.
3. `tcp` Dieser Filter wurde verwendet, um die TCP-Pakete anzuzeigen, die während der Kommunikation ausgetauscht wurden. Dadurch konnten der Aufbau der TCP-Verbindung und die Übertragung der HTTP-Pakete über TCP analysiert werden.
4. `http2.stream_id == ...`: Dieser Filter wurde verwendet, um die HTTP/2-Pakete anzuzeigen, die zum einem bestimmten Stream gehören. So kann beispielsweise nur der Stream mit der Anfrage oder nur der Stream mit dem Einstellungsaustausch angezeigt werden, um die Kommunikation in diesem Stream genauer zu analysieren.

Nach dem Filtern der Frames wurden mit Wireshark die Details der einzelnen Pakete analysiert. Dafür wurde das Aufklappen der entsprechenden Netzwerkschicht genutzt, so sind Details wie Größe der Pakete, Flags, Header und Payload sichtbar, um die Kommunikation zwischen Client und Server zu verstehen. 1

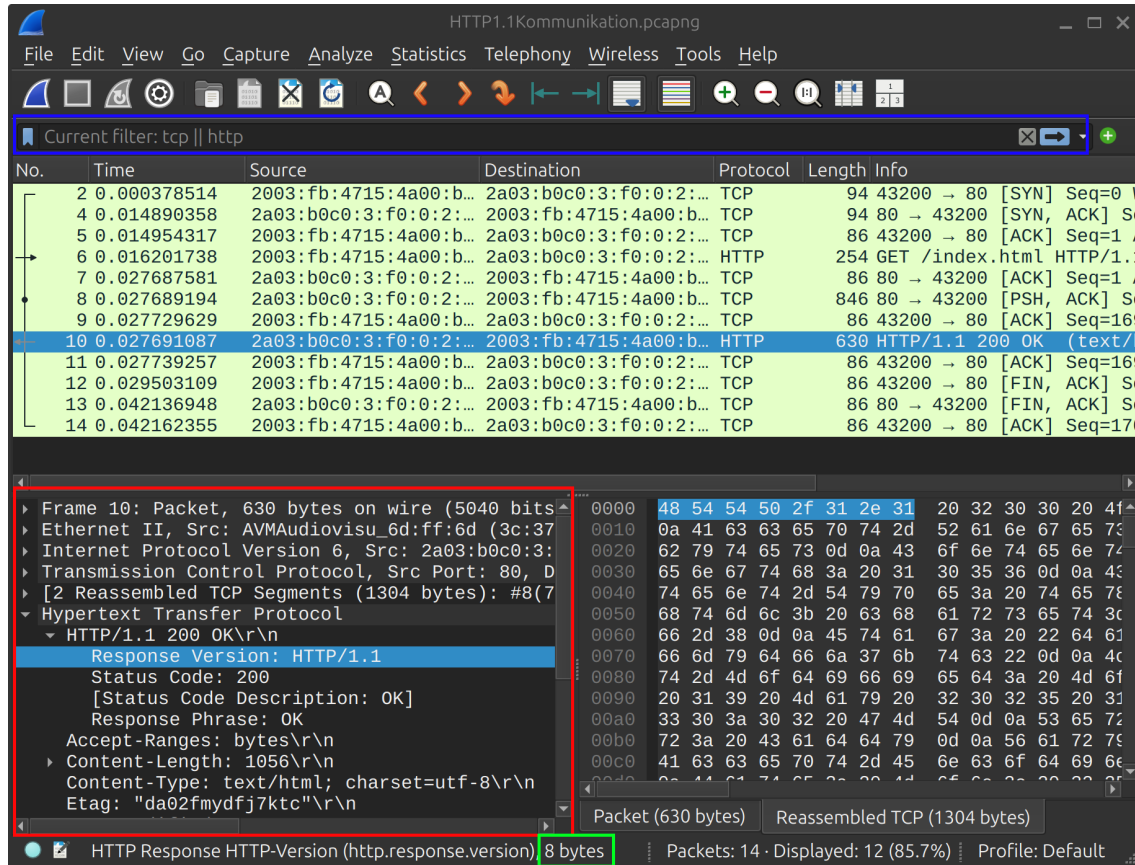


Abbildung 1: Analyse der HTTP-Kommunikation in Wireshark. In der roten Box befindet sich der Bereich in dem die Netzwerkschichten genauer betrachtet werden können, um Details wie Flags, Header und Payload zu analysieren. In der grünen Box die Größe des ausgewählten Teils des Paketes. Und in der blauen Box die Filter, um die relevanten Pakete zu isolieren.

1.3 Analyse der HTTP/2-Kommunikation

Über Wireshark können die versendeten Pakete im Detail nachverfolgt werden. Die Kommunikation beginnt dabei mit dem Aufbau einer TCP-Verbindung. Der Client fragt die TCP-Verbindung zum Server an, und der Server antwortet mit einem SYN-ACK-Paket. Der Client bestätigt den Verbindungsaufbau mit einem ACK-Paket, wodurch die TCP-Verbindung etabliert wird. Nach dem erfolgreichen Aufbau der TCP-Verbindung wird diese für die HTTP/2-Kommunikation genutzt, um Anfragen und Antworten zwischen Client und Server auszutauschen. Dabei sendet der Client ein Connection Preface, Settings und Window Update. Das Connection Preface ist ein einzigartiger String, der dem Server signalisiert, dass der Client HTTP/2 unterstützt, während die Settings und Window Update Pakete die Parameter für die Kommunikation festlegen. Die Settings 1 enthalten Informationen über die unterstützten Funktionen und Einstellungen des Clients, während die Window Update Pakete 2 die Flusskontrolle für die Datenübertragung regeln.

Setting	Value
Header Table Size	4096
Enable PUSH	0
Initial Window Size	65535
Max Frame Size	16384
Max Concurrent Streams	100
Max Header List Size	65536

Tabelle 1: HTTP/2 Settings Parameter

Connection Window	Size
Before	65535
After	16842751

Tabelle 2: Initiale Connection Window Sizes

Für die ganze HTTP/2-Kommunikation wurden insgesamt 7 Frames auf der Verbindungsschicht verschickt, diese 7 Frames enthalten 10 HTTP/2-Frames sowie das Connection Preface, welches gemäß RFC 9113 §3.4 [6] kein HTTP/2-Frame ist, von welchen 5 HTTP/2-Frames gesendet und 5 empfangen wurden. Dabei wurden insgesamt 1419 Bytes übertragen, von welchen 161 Bytes gesendet und 1258 Bytes empfangen wurden. 3

Bei der beobachtbaren HTTP/2-Kommunikation werden Settings nur über Stream 0 gesendet, während Stream 1 hier für die Get Anfrage genutzt wird. Außerdem werden Flags wie End Headers, End Stream, Padded und Priority in den Header Frames verwendet, um die Struktur und Priorität der übertragenen Daten zu steuern. Die Nutzlastgrößen variieren je nach Frame-Typ, wobei die Data Frames die größte Nutzlast aufweisen, da sie die eigentlichen Daten der HTTP-Anfrage und -Antwort enthalten. Eine weitere interessante Flag ist die ACK-Flag in den Settings Frames. Die ACK-Flag wird genutzt um das Erhalten der zuvor gesendeten Settings und deren Anwendung zu bestätigen, dabei enthält dieser HTTP/2-Frame nie eine Nutzlast [6]. 4

HPACK

HPACK ist ein Header-Komprimierungsverfahren, das in HTTP/2 verwendet wird, um die Größe der übertragenen Header zu reduzieren und somit die Effizienz der Kommunikation zu verbessern. Außerdem bietet es durch die Komprimierung der Header eine verbesserte Sicherheit, da es die Angriffsfläche für bestimmte Arten von Angriffen reduziert [5]. Die Komprimierung erfolgt durch die Verwendung von statischen und dynamischen Tabellen, die die Header-Felder und deren Werte speichern. Jeder Endpunkt verwaltet zwei dynamische Tabellen nach dem First-In-First-Out (FIFO) Prinzip, eine für das Kodieren und eine für das Dekodieren. Die Größe der dynamischen Tabellen kann durch die Settings Frames angepasst werden, um die Effizienz der Komprimierung zu optimieren. In unserem Fall wurde die Header table size auf 4096 Bytes eingestellt, was bedeutet, dass die dynamische Tabelle bis zu 4096 Bytes an Header-Feldern und Werten speichern kann. Die statische Tabelle enthält eine vordefinierte Liste von häufig verwendeten Header-Feldern und Werten, die von beiden Endpunkten geteilt wird. Die dynamische Tabelle wird während der Kommunikation aktualisiert, um neue Header-Felder und Werte aufzunehmen, die nicht in der statischen Tabelle enthalten sind. Dabei gibt es verschiedene Szenarien. Wir betrachten immer ein Key-Value Paar, also ein Header-Feld und dessen Wert. Dabei kann entweder der Key in der dynamischen Tabelle vorhanden sein, oder die Kombination aus Key und Value, oder keines von beiden. Wenn etwas in der dynamischen Tabelle vorhanden ist, wird ein Index verwendet, so kann auf das entsprechende Key-Value-Paar verwiesen werden, was die Größe der übertragenen Header reduziert. [5]

Beispielhaft kann Paket 7 betrachtet werden (Die HTTP/2 Header Frame mit der Get Anfrage). In diesem Paket wird die Get Anfrage gesendet, dabei ist der Key *:method* mit dem Wert *GET* in der statischen Tabelle vorhanden. Damit ist für die Übertragung nur 1 Byte notwendig, während die

Paketnummer	Empfangen oder gesendet	HTTP/2-Frame Größe in Bytes
6	Gesendet	24
6	Gesendet	45
6	Gesendet	13
7	Gesendet	57
7	Gesendet	13
10	Empfangen	45
12	Gesendet	9
13	Empfangen	9
13	Empfangen	13
14	Empfangen	126
16	Empfangen	1065
Gesamt Gesendet		161
Gesamt Empfangen		1258
Gesamt		1419
Frames Gesamt		7
HTTP/2-Frames Gesamt		10

Tabelle 3: Gesendete und empfangene HTTP/2-Frames mit ihrer Größe in Bytes. Ohne Connection Preface, da dies im Standard [6] explizit nicht als Frame betrachtet wird

Paketnummer	Pakettyp	Stream-ID	Flags	Nutzlastgrößen (Bytes)
6	Connection Preface	-	-	0
6	Settings	0	ACK=False	36
6	Window Update	0	-	4
7	Headers	1	End Headers=True, End Stream=True, Padded=False, Priority=False	48
7	Window Update	1	-	4
10	Settings	0	ACK=False	24
12	Settings	0	ACK=True	0
13	Settings	0	ACK=True	0
13	Window Update	0	-	4
14	Headers	1	Priority=False, End Headers=True, End Stream=False, Padded=False	117
16	Data	1	End Stream=True, Padded=False	1056

Tabelle 4: HTTP/2-Paketdetails

Übertragung des Key-Value Paares ohne Komprimierung mindestens 11 Bytes benötigen würde.

Im selben Paket wird der Key *user-agent* mit dem Wert *python-httpx/0.28.1* übertragen, dabei ist der Key in der statischen Tabelle vorhanden, aber die Kombination aus Key und Value nicht. Daher wird ein Index für den Key *user-agent* verwendet, aber der Wert *python-httpx/0.28.1* wird nicht indiziert und anders komprimiert übertragen, was insgesamt 16 Bytes benötigt.

1.4 Analyse der HTTP/1.1-Kommunikation

Die HTTP/1.1-Kommunikation wurde analog zu der HTTP/2-Kommunikation aufgenommen 1.2.

Der Aufbau der HTTP/1.1-Pakete ist wie nach dem Standard zu erwarten 3.

```

1 HTTP-message    = start-line CRLF
2                  *( field-line CRLF )
3                  CRLF
4                  [ message-body ]

```

Code 3: HTTP/1.1 Message Format [2]

Dabei besteht die HTTP/1.1-Nachricht aus einem Start Line, gefolgt von Headern und einem optionalen Body. Die Start Line besteht aus einer Method, einem Request Target und einer HTTP-Version. Die Header bestehen aus einem Header Name und einem Header-Value getrennt durch einen Doppelpunkt, die Tabelle 5 zeigt den Aufbau der Get Anfrage in der pcapng Datei.

Start Line	GET /index.html HTTP/1.1\r\n
Header Name	Header Value
Host	cuil.internet-transparency.org
Accept	*/*
Accept-Encoding	gzip, deflate
Connection	keep-alive
User-Agent	python-httpx/0.28.1

Tabelle 5: HTTP/1.1 Get Anfrage in der pcapng Datei

Der Payload bei der Anfrage ist 0 Bytes groß, da es sich um eine Get Anfrage handelt, die nur aus dem Header besteht.

Start Line	HTTP/1.1 200 OK\r\n
Header Name	Header Value
Accept-Ranges	bytes
Content-Length	1056
Content-Type	text/html; charset=utf-8
Etag	da02fmydfj7ktc
Last-Modified	Mon, 19 May 2025 10:30:02 GMT
Server	Caddy
Vary	Accept-Encoding
Date	Mon, 25 May 2026 15:16:14 GMT
Body	1056 Bytes

Tabelle 6: HTTP/1.1-Antwort in der pcapng Datei

Die HTTP/1.1-Antwort besteht aus einem Start Line, gefolgt von Headern und einem Body. Dabei ist der Payload (Body) 1056 Bytes groß, wie im Content-Length Header angegeben 6. Die HTTP/1.1-Kommunikation in der pcapng Datei entspricht somit dem Standard und es gibt keine Auffälligkeiten oder Besonderheiten anzumerken.

Für die ganze Kommunikation mit dem Server wurden 12 Frames auf der Verbindungsschicht ausgetauscht, davon waren 10 TCP-Pakete und 2 HTTP-Pakete. Für die HTTP/1.1-Anfrage wurde 1 HTTP-Paket gesendet und für die HTTP/1.1-Antwort wurde 1 HTTP-Paket empfangen. Die Anfrage war 168 Bytes groß. Die Antwort war 1304 Bytes groß, wobei Header und Body des HTTP/1.1 über 2 TCP-Segmente übertragen wurden.

DNS-Cache in der pcapng

Unabhängig von der HTTP/1.1-Kommunikation fällt in der pcapng Datei eine Besonderheit auf. Die TCP-Verbindung zum Server wird über IPv6 gestartet bevor die Antwort des DNS-Servers die URL in eine IPv6-Adresse aufgelöst hat. Das ist insofern ungewöhnlich, als der Client die Adresse hinter der URL eigentlich erst nach der DNS-Abfrage kennen sollte, um die TCP-Verbindung zum Server zu starten. Die wahrscheinlichste Erklärung ist, dass die IP-Adresse bereits in einem der DNS-Caches lag. DNS-Anfragen werden auf vielen Ebenen gecacht, wie zum Beispiel im Betriebssystem, im Browser, Router oder in einem lokalen DNS-Server. Wobei in unserem Fall wahrscheinlich der DNS-Cache des Betriebssystems die IP-Adresse vorhielt, da kein Browser genutzt wurde und Kommunikation mit dem lokalen DNS-Server oder dem Router offensichtlich nicht abgewartet wurde.

1.5 Vergleich der HTTP/1.1- und HTTP/2-Austausche

In diesem Abschnitt werden die beiden HTTP-Austausche miteinander verglichen, um die Unterschiede in der Anzahl der gesendeten Bytes auf der Verbindungsschicht und der gesendeten HTTP-Bytes zu analysieren.

Übertragene Bytes und Pakete

HTTP/2 nutzt im Vergleich zu HTTP/1.1 Header-Komprimierung und teilt Nachrichten in mehrere HTTP-Frames auf.

Absender	HTTP1	HTTP2
Client (Link Layer Frames)	778 Bytes	943 Bytes
Server (Link Layer Frames)	1742 Bytes	1954 Bytes
Client (HTTP)	168 Bytes	161 Bytes
Server (HTTP)	1304 Bytes	1258 Bytes
TCP-Pakete Insgesamt	12	17

Tabelle 7: Anzahl der gesendeten Bytes auf der Verbindungsschicht im HTTP/1.1- und HTTP/2-Austausch inklusive TCP-Paketen und nur HTTP betrachtet.

Basierend auf der Tabelle 7 kann festgestellt werden, dass HTTP/2 im Vergleich zu HTTP/1.1 mehr Bytes auf der Verbindungsschicht sendet, sowohl vom Client als auch vom Server. Dies liegt daran, dass HTTP/2 zusätzliche Frames auf der Verbindungsschicht verwendet, um die Kommunikation einzuleiten und Einstellungen auszuhandeln. Die Anzahl der gesendeten HTTP-Bytes ist jedoch bei HTTP/2 geringer, was auf die Header-Komprimierung zurückzuführen ist. Insgesamt zeigt dies, dass HTTP/2 effizienter in der Übertragung von HTTP-Daten ist, obwohl es initial mehr Bytes auf der Verbindungsschicht sendet. Dies lässt sich auch bei der Anzahl ausgetauschter TCP-Pakete beobachten. HTTP/2 benötigt mehr TCP-Pakete, um die Kommunikation zu etablieren und aufrechtzuerhalten, da Window Updates und Settings ausgetauscht werden. Im Gegensatz dazu benötigt HTTP/1.1 weniger TCP-Pakete, da es keine zusätzlichen Frames für die Verbindungsaushandlung verwendet.

Verbindungsaufbau Overhead

Der Overhead beider Protokolle kann durch die Anzahl der gesendeten Bytes auf der Verbindungsschicht im Vergleich zu den gesendeten HTTP-Bytes bewertet werden. Bei der HTTP/1.1-Kommunikation wurden 778 Bytes auf der Verbindungsschicht gesendet, während nur 168 Bytes HTTP-Daten übertragen wurden, damit wurden etwa **21,6 %** der gesendeten Bytes für die eigentlichen HTTP-Daten verwendet. Im Gegensatz dazu wurden bei HTTP/2 943 Bytes auf der Verbindungsschicht gesendet, während nur 161 Bytes HTTP-Daten übertragen wurden, was etwa **17,1 %** der gesendeten Bytes für die HTTP-Daten entspricht. Bei dem Vergleich muss jedoch beachtet werden, dass HTTP/2 zusätzliche Frames für die Verbindungsaushandlung und Einstellungen verwendet, was zu einem höheren Overhead auf der Verbindungsschicht führt, aber gleichzeitig die Effizienz der HTTP-Datenübertragung verbessert. Deshalb nutzt HTTP/2 auf Anwendungsebene nur 57 Bytes für die Get-Anfrage, während HTTP/1.1 168 Bytes für die gleiche Anfrage benötigt. Außerdem benötigt HTTP/2 263 Bytes, gesendet vom Client, für den initialen Settings Austausch. Dies bedeutet auch, dass HTTP/2 immer effizienter wird je mehr Anfragen über die gleiche Verbindung gesendet werden. Der initiale Verbindungs-overhead wird auf mehrere Anfragen verteilt, jede weitere Anfrage ist aber deutlich effizienter als bei HTTP/1.1, da die Header-Komprimierung und die Multiplexing-Fähigkeiten von HTTP/2 genutzt werden können.

HTTP/2-Multiplexing

Ein weiterer wichtiger Unterschied zwischen HTTP/1.1 und HTTP/2 ist die Fähigkeit von HTTP/2, mehrere Anfragen gleichzeitig über die gleiche Verbindung zu multiplexen. HTTP/2 nutzt dafür Streams. Dabei läuft eine Anfrage pro Stream. Die Daten werden in Frames bestimmter Größe aufgeteilt, die unabhängig voneinander gesendet und empfangen werden können. Dabei bleibt die Reihenfolge der Frames innerhalb eines Streams erhalten, aber die Frames verschiedener Streams können in beliebiger Reihenfolge gesendet werden (Multiplexing). Das verhindert, dass eine Anfrage die Bearbeitung einer anderen blockiert, da die Frames verschiedener Streams verzahnt voneinander gesendet und empfangen werden können. Der Empfänger ordnet die Frames dann entsprechend der Stream-IDs. Ein Stream kann also als logische Verbindung innerhalb der TCP-Verbindung betrachtet werden, die es ermöglicht, mehrere Anfragen gleichzeitig zu bearbeiten, ohne dass sie sich gegenseitig blockieren.

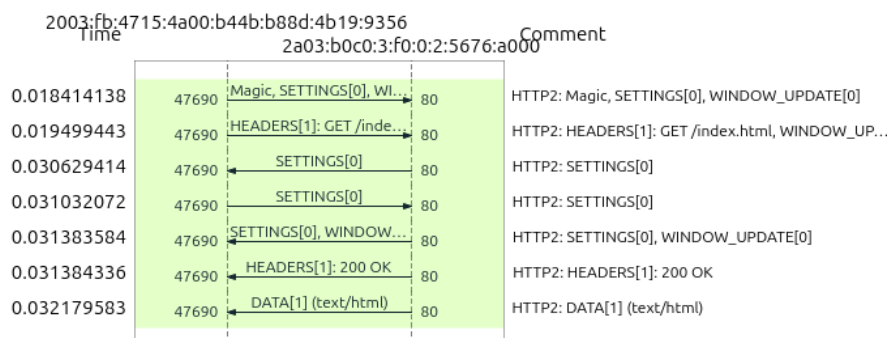


Abbildung 2: Stream 0 wird für die initiale Verbindungsaushandlung verwendet, während Stream 1 für die GET-Anfrage genutzt wird.

In unserem HTTP/2-Austausch 2 wurden 7 Link Layer Frames mit HTTP/2-Protokoll gesendet. Dabei wurde erst der Settings-Austausch über Stream 0 initiiert, bevor die GET-Anfrage über Stream 1 gesendet wurde. Nachdem dann die Kommunikation über Stream 0 abgeschlossen wurde, wurde die Response der GET-Anfrage über Stream 1 empfangen. Multiplexing war hier nicht in größerem Stil notwendig und auch nicht möglich, da der Settings-Austausch zwingend erfolgt sein muss, bevor Antworten gesendet werden. Sonst fehlen Kommunikationsparameter für die Streams.

1.6 Ablaufbeschreibung eines HTTP/2-Clients über h2c

Settings Austausch

1. TCP-Handshake
 - Client sendet SYN → Verbindungsanfrage (Pakete 3)
 - Server antwortet mit SYN-ACK → Verbindungsbestätigung (Pakete 4)
 - Client bestätigt mit ACK → Verbindung steht (Pakete 5)
2. HTTP/2 Connection Preface → Information über die HTTP/2-Unterstützung (Paket 6)
3. SETTINGS Frame → Konfiguration der Verbindung (Paket 6)
4. Window Update Frame → Erlaubt dem Server, mehr Daten zu senden (Paket 6)
5. Server sendet SETTINGS Frame (Paket 10)
6. Client bestätigt SETTINGS Frame (Paket 12)
7. Server bestätigt SETTINGS Frame (Paket 13)

HPACK-Kodierung

1. Client sendet HTTP/2 HEADERS Frame mit der HPACK-kodierten Anfrage und nutzt dafür seine Kodierungstabelle
2. Server empfängt HEADERS Frame
3. Server dekodiert HPACK-kodierte Header
4. Server aktualisiert die dynamische Dekodierungstabelle mit den neuen Headern
5. Server verarbeitet die Anfrage und bereitet die Antwort mit HPACK-Kodierung mit der dynamischen Kodierungstabelle vor

Antworten Verarbeiten

Der Client wartet bis alle Frames mit den entsprechenden Stream-IDs empfangen wurden und setzt die HTTP-Nachricht zusammen. Sobald die Nachricht vollständig ist, verarbeitet der Client die Antwort entsprechend der HTTP/2-Spezifikation, z.B. durch Anzeigen der Webseite oder Speichern der Daten.

Streams verwalten

Jede neue Anfrage öffnet einen neuen Stream mit einer neuen Stream-ID. Ein Thread könnte pro Stream laufen und den HTTP/2-Frames erstellen. Dann wird auf der anderen Seite das Paket empfangen und basierend auf der Stream-ID zugeordnet und verarbeitet. Sobald die Antwort vollständig ist, wird der Stream geschlossen.

1.7 Reflexion und Diskussion

Die Analyse der HTTP-Kommunikation stellte wenige Herausforderungen dar. Die Unterscheidung zwischen Frames auf der Verbindungsschicht und HTTP/2-Frames war zunächst unklar, da mehrere HTTP/2-Frames in einem TCP-Paket übertragen werden können. Zudem hat die Klassifikation des Connection Preface Recherche gefordert, da es kein HTTP/2-Frame ist, sondern ein Magic String, der vor den Frames gesendet wird.

HTTP/2 hat im Vergleich zu HTTP/1.1 einen Overhead bei der Initialisierung der Verbindung. Nach diesem Overhead werden aber die Vorteile von HTTP/2 deutlich. Durch HPACK sind die

Header deutlich kleiner, was die Übertragung beschleunigt. Außerdem ermöglicht HTTP/2 das Multiplexing von Anfragen, wodurch die Latenz speziell bei vielen parallel laufenden Anfragen deutlich reduziert wird. Insgesamt zeigt sich, dass HTTP/2 deutlich schneller ist als HTTP/1.1, insbesondere bei vielen Anfragen oder großen Headern. Die Ergebnisse bestätigen die theoretischen Vorteile von HTTP/2 und zeigen, dass die Implementierung von HTTP/2 in modernen Webbrowsern und Servern gerechtfertigt ist, um die Leistung und Benutzererfahrung im Web zu verbessern. Moderne Webanwendungen profitieren von HTTP/2, da sie oft viele Ressourcen (Bilder, Skripte, Stylesheets) laden müssen, und HTTP/2 ermöglicht es, diese effizienter zu übertragen. Ein Großteil der Seite kann bereits angezeigt werden, während die restlichen Ressourcen noch geladen werden, was die wahrgenommene Ladezeit verbessert. Bei diesen Vorteilen fällt die anfängliche Verzögerung durch die Verbindungseinrichtung von HTTP/2 kaum ins Gewicht, zumal der Unterschied sogar im Direktvergleich mit HTTP/1.1 nicht für einen Nutzer spürbar ist.

2 SMTP

Für die Durchführung der SMTP Kommunikation wurde das Programm `telnet` (Version 308) auf einem MacBook Air mit Apple M1-Chip unter macOS 26.5 verwendet. Die Verbindung zum Server der Universität Münster wurde über das Uni-VPN (Cisco Secure Client) mit dem folgenden Befehl hergestellt:

```
1 telnet udc-m-wwu1.uni-muenster.de 25
```

Code 4: Telnet Befehl zum SMTP Verbindungsaufbau

Der Server antwortete mit 220 und bestätigte ESMTP als erweitertes Protokoll. Anschließend wurden die SMTP Befehle vorbereitet und der Austausch durchgeführt.

Erster Versuch

Im ersten Versuch wurden alle SMTP Befehle als zusammenhängender Block in Telnet eingefügt. Der Server akzeptierte die Anfrage und bestätigte die Zustellung mit 250 `ok: Message 406218478 accepted` 3.

```
linushenn@vpn-pool1-pnt-16313 ~ % telnet udc-m-wwu1.uni-muenster.de 25
Trying 128.176.118.7...
Connected to udc-m-wwu1.uni-muenster.de.
Escape character is '^]'.
220 UDCM-WWU1.UNI-MUENSTER.DE ESMTP
EHLO monkey.banana.com
MAIL FROM:<linus.henn@uni-muenster.de>
RCPT TO:<nkempen@uni-muenster.de>
DATA
From: Linus Henn <linus.henn@uni-muenster.de>
To: nkempen@uni-muenster.de
Date: Sat, 01 Jan 2000 00:00:00 +0000
Subject: CUIL Minilab 1

Hallo, ich hoffe das kommt an.

.
QUIT250-UDCM-WWU1.UNI-MUENSTER.DE
250-8BITMIME
250-SIZE 26214400
250 STARTTLS
250 sender <linus.henn@uni-muenster.de> ok
250 recipient <nkempen@uni-muenster.de> ok
354 go ahead
250 ok: Message 406218478 accepted
^]
telnet> quit
Connection closed.
```

Abbildung 3: Erster SMTP-Versuch: Alle Befehle als Block eingefügt.

Zweiter Versuch

Nach erneuter Recherche anhand von RFC 5321 §4.3 wurde festgestellt, dass der Client nach jedem gesendeten Befehl auf die Antwort des Servers warten muss, bevor der nächste Befehl gesendet wird [4]. Im zweiten Versuch wurden die Befehle daher einzeln geschickt.

Befehl	Serverantwort	Bedeutung
(telnet udcM-wwu1... 25)	220 UDCM-WWU1... ESMTP	Server bereit, ESMTP unterstützt
EHLO monkey.banana.com	250- Capability-Liste	Client identifiziert sich, Server listet Erweiterungen
MAIL FROM:<linus.henn@...>	250 sender ok	Absender akzeptiert
RCPT TO:<nkempen@...>	250 recipient ok	Empfänger akzeptiert
DATA	354 go ahead	Server bereit für Nachrichteninhalt
(Header + Body) + .	250 ok	Nachrichte zugestellt/akzeptiert
QUIT	221 UDCM-WWU1...	Verbindung korrekt beendet

Tabelle 8: SMTP Befehle im zweiten Versuch

Der Server warb in der EHLO-Response mit den Erweiterungen 8BITMIME, SIZE 26214400 und STARTTLS. Der Server antwortete auf QUIT mit 221 und schloss die Verbindung, was dem korrekten Verbindungsabbau gemäß RFC 5321 §4.1.1.10 entspricht [4].

```

linushenn@vpn-pool1-pnt-16313 ~ % telnet udcM-wwu1.uni-muenster.de 25
Trying 128.176.118.7...
Connected to udcM-wwu1.uni-muenster.de.
Escape character is '^]'.
220 UDCM-WWU1.UNI-MUENSTER.DE ESMTP
EHLO monkey.banana.com
250-UDCM-WWU1.UNI-MUENSTER.DE
250-8BITMIME
250-SIZE 26214400
250 STARTTLS
MAIL FROM:<linus.henn@uni-muenster.de>
250 sender <linus.henn@uni-muenster.de> ok
RCPT TO:<nkempen@uni-muenster.de>
250 recipient <nkempen@uni-muenster.de> ok
DATA
354 go ahead
From: Linus Henn <linus.henn@uni-muenster.de>
To: nkempen@uni-muenster.de
Date: Sat, 01 Jan 2000 00:00:00 +0000
Subject: CUIL Minilab 1

Hallo, Test 2.

.
250 ok: Message 406219343 accepted
QUIT
221 UDCM-WWU1.UNI-MUENSTER.DE
Connection closed by foreign host.

```

Abbildung 4: Zweiter SMTP Versuch: Korrekte Befehlsfolge

Auswertung

Der Erfolg des ersten Versuchs könnte sich durch das Konzept des *SMTP Pipelinings* erklären. Pipelining wird als das Senden mehrerer SMTP-Befehle, ohne auf die jeweiligen Serverantworten zu Warten bezeichnet [3].

Der Grund könnte zudem in der zugrundeliegenden TCP-Verbindung liegen: TCP garantiert für alle Sendebytes Zuverlässigkeit und richtige Reihenfolge im Empfangspuffer. Der SMTP Server liest und verarbeitet die Befehle sequenziell aus dem Puffer.

Obwohl Pipelining hier funktionierte, ist dieses Verhalten ohne explizite Serverunterstützung nicht garantiert. Der Server dieses Versuchs warb nicht mit der Erweiterung PIPELINING nach RFC 2920 [3]. Somit wird keine Garantie für korrektes Verhalten bei gepipelinten Befehlen gegeben: Schlägt ein früher Befehl fehl und befinden sich nachfolgende Befehle aufgrund des Pipelinings bereits im Empfangspuffer, kann es zu undefinierten Zuständen kommen. Pipelining gibt in diesem Fall Garantie für sicheres Recovery [3].

Literatur

- [1] Encode OSS. Http/2 support in httpx, 2026. URL: <https://www.python-httpx.org/http2/>.
- [2] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP/1.1. RFC 9112, June 2022. URL: <https://www.rfc-editor.org/info/rfc9112>, doi:10.17487/RFC9112.
- [3] Ned Freed. SMTP Service Extension for Command Pipelining. RFC 2920, September 2000. URL: <https://www.rfc-editor.org/info/rfc2920>, doi:10.17487/RFC2920.
- [4] John C. Klensin. Simple Mail Transfer Protocol. RFC 5321, October 2008. URL: <https://www.rfc-editor.org/info/rfc5321>, doi:10.17487/RFC5321.
- [5] Roberto Peon and Herve Ruellan. HPACK: Header Compression for HTTP/2. RFC 7541, May 2015. URL: <https://www.rfc-editor.org/info/rfc7541>, doi:10.17487/RFC7541.
- [6] Martin Thomson and Cory Benfield. HTTP/2. RFC 9113, June 2022. RFC 9113, Section 4.3.1 "Compression State", accessed: 2026-05-24. URL: <https://www.rfc-editor.org/info/rfc9113>, doi:10.17487/RFC9113.